

Maior Subsequência Comum

Descrição da solução

O programa recebe duas sequências de tickers de ações da B3, representadas por dois vetores de Strings com tamanhos M e N. O objetivo é encontrar a maior subsequência que aparece nas duas colunas, mantendo a mesma ordem em que os tickers aparecem em cada uma delas.

Para uma sequência ser considerada válida, os tickers precisam aparecer nas duas corretoras.

A solução começa lendo os valores M e N e criando os dois vetores. Em seguida, os tickers são armazenados nos vetores das respectivas corretoras.

Para encontrar os elementos em comum, são utilizados dois 'for'. O primeiro percorre a primeira coluna e o segundo percorre a segunda coluna. Quando dois tickers são iguais, suas posições nas duas sequências são identificadas.

Depois disso, é utilizada uma busca recursiva para continuar procurando outros tickers que possam fazer parte da coluna. A busca recebe como referência as últimas posições utilizadas nas duas colunas. Ou seja, um novo ticker só pode ser escolhido se estiver depois das posições anteriores nas duas corretoras.

A sequência que está sendo testada é armazenada em uma lista chamada de 'atual'

Quando um *ticker* válido é encontrado, ele é adicionado à coluna. O algoritmo então continua a busca a partir das novas posições.

Depois, o último ticker escolhido é removido da coluna atual. Dessa forma, o programa consegue voltar para uma situação anterior e testar outras possibilidades. Realizando o backtracking.

Durante a execução, também é mantida uma lista contendo a melhor subsequência encontrada até aquele momento. Quando uma sequência maior é encontrada, ela passa a ser armazenada como a 'melhor' solução.

Ao final da execução, o programa apresenta o tamanho da maior subsequência encontrada e os tickers que fazem parte dela.

Modelagem

A primeira sequência é representada pelo vetor corretoraA e a segunda sequência pelo vetor corretoraB. Cada posição dos vetores representa um ticker.

A busca utiliza duas posições para controlar a ordem dos elementos. A última posição utilizada na primeira sequência e a última posição utilizada na segunda sequência são passadas para a próxima etapa da busca.

A sequência atual representa os tickers escolhidos durante um determinado caminho da busca. A melhor sequência representa a maior subsequência encontrada entre todos os caminhos analisados.

A busca começa com as posições anteriores definidas como menos um, pois inicialmente nenhum ticker foi escolhido. Assim, o primeiro elemento pode ser procurado desde a posição zero.

Complexidade de tempo (O):

Pior Caso: $T(M,N) = O(M \times N \times 2^N)$

Se $M = N \rightarrow O(N^2 \cdot 2^N)$.

Melhor Caso: $T(M,N) = O(M \times N)$

Se $M = N \rightarrow O(N^2)$

Essa implementação utiliza uma busca recursiva para explorar as possíveis subsequências viáveis. Em cada chamada podem ser percorridos pares de elementos das duas listas, com custo de até **$O(MN)$** e a quantidade de possibilidades cresce exponencialmente. Por isso, considerando o pior caso estou utilizando **$O(M \times N \times 2^N)$** como limite superior simplificado. Quando M e N são iguais isso fica **$O(N^2 \cdot 2^N)$** .

O fator M vezes N está relacionado às comparações entre os elementos das duas sequências, enquanto o fator exponencial está relacionado à exploração das diferentes possibilidades durante o processo de backtracking.

Por isso o tempo de execução pode aumentar conforme os valores de M e N aumentam.

Complexidade de espaço

Os dois vetores utilizados para armazenar as entradas ocupam espaço proporcional a:

M + N.

Além disso, a sequência atual pode armazenar uma quantidade de elementos proporcional ao tamanho da menor sequência, assim como a profundidade da chamada recursiva.

Considerando os vetores de entrada e as estruturas utilizadas durante a execução, a complexidade de espaço é $O(M + N)$.

Análise experimental

Para analisar o comportamento da solução, podem ser realizados testes aumentando gradualmente os valores de M e N. Os tempos de execução devem ser medidos durante os testes e registrados para comparar o comportamento do programa conforme o tamanho das entradas aumenta.

É esperado que o tempo de execução aumente significativamente em entradas maiores, principalmente quando existem muitos tickers em comum, pois nesse caso o algoritmo precisa explorar uma quantidade maior de possibilidades.

Análise experimental

Para analisar o comportamento do programa, podem ser realizados testes aumentando gradualmente os valores de M e N

Uma expectativa para essa sequência considerando o **pior** caso:

Entrada M = N	Operações $N^2 \times 2^N$	Crescimento relativo
5	800	800
10	102.400	102 mil
15	7.372.800	7,4 milhões

20	419.430.400	419 milhões
25	20.971.520.000	21 bilhões
30	966.367.641.600	966 bilhões
35	42.177.300.480.000	42,2 trilhões
40	1.759.218.604.441.600	1,76 quatrilhão
45	72.843.167.922.779.136	72,8 quatrilhões
50	72.843.167.922.779.136	281,5 quatrilhões
55	1.981.252.112.111.972.864. 000	1,98 sextilhão
60	13.322.419.670.454.415.36 0.000	13,3 sextilhões

Considerando o melhor caso:

Entrada $M = N$	Operações N^2	Crescimento
5	25	25

10	100	100
15	225	225
20	400	400
25	625	625
30	900	900
35	1225	1225
40	1600	1600
45	2025	2025
50	2500	2500
55	3025	3025
60	3600	3600

A expectativa é que o tempo aumente de forma significativa conforme o tamanho das entradas aumenta, principalmente quando existem muitos tickers em comum, pois isso gera mais possibilidades para a busca explorar.

Uso de Inteligência Artificial

Foi utilizada Inteligência Artificial como ferramenta de apoio durante o desenvolvimento do trabalho. Foi utilizada principalmente para auxiliar na compreensão e revisão de conceitos de programação em Java, como vetores, ArrayList, recursão e backtracking.